Multi-Agent Chat Simulation (beta v1)

A browser-based chat application where you have a conversation with several AI-driven characters at the same time. The characters each have a distinct personality and role in a shared scenario, and a behind-the-scenes "orchestrator" decides, after each message you send, which characters should reply and in what order.

`config.yaml` is the main file for changing the application. All wording of the study screens (including questions, answers and button captions), all agent descriptions and prompts, and all settings are defined there. `main.py` only contains the mechanics (what happens when a button is clicked).

What Does It Do?

When you open the app in your browser you see a login screen. After entering your name (and optionally a password), you are guided through the study screens (see Study Flow) and finally enter the live group chat. You type messages just like in any chat app. The AI agents read what you wrote, consider the ongoing conversation, and respond -- often multiple agents in a row, sometimes just one, sometimes none (if the conversation naturally calls for silence).

The experience is designed to feel like a real team meeting or group discussion, not a one-on-one chatbot.

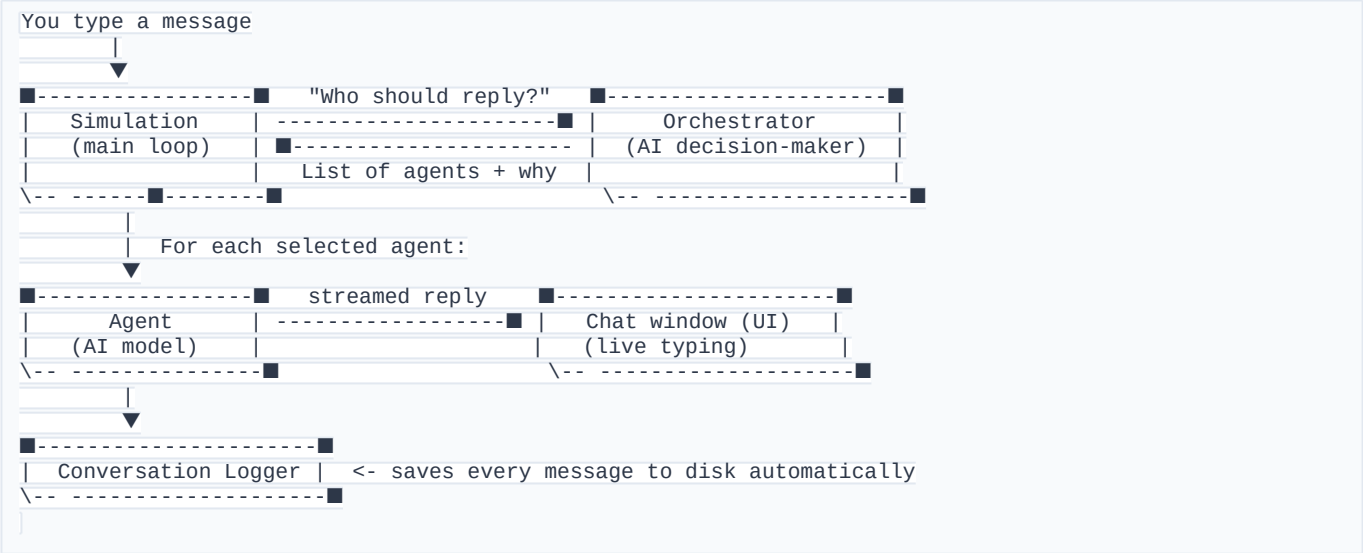
Study Flow

After the login screen, the participant moves through 10 screens. A progress bar at the top lets the participant jump back to any screen that was already completed.

#	Screen	What happens
1	Consent	The participant reads the information and ticks the consent box before continuing.
2	Briefing	Shows the task (`ui.task_description`) and the participant's role as team leader.
3	Check 1	Comprehension questions about the task. All must be correct; otherwise the participant is sent back to the briefing.
4	Meet team	Cards for the Coordinator and the other agents (names, titles and descriptions from the `agents` section). The order of the specialist cards is randomised (counterbalanced).
5	Check 2	Comprehension questions about the team. All must be correct; otherwise the participant is sent back to the team page.
6	Tutorial	Four practice steps: message the whole team, `@mention` one agent, ask the image agent for an image, and pause/resume.
7	Role	One of the leadership-role conditions is assigned at random; some conditions require a written reflection.
8	Check 3	A 7-point Likert reflection scale; every item must be answered.
9	Live task	The multi-agent chat with a Task/Agents/Tips side panel, elapsed timer, `@mention` buttons, Pause/Resume and poster saving.
10	Debrief	Thank-you message and a link to the exit survey that includes the session ID.

Every step is written to an event log (shown at the bottom of the page and saved with the conversation transcript).

How It Works



1. **You** type a message and press Send (or Enter).
2. The **Simulation** passes the full conversation history to the **Orchestrator**.
3. The **Orchestrator** is itself an AI model. It reads the conversation and gives each agent a "response probability" based on the agent's role, how often they have spoken, and how relevant they are to what you just said. It then picks which agents should reply.
4. Each selected **Agent** generates a reply, which appears word-by-word in the chat window (live-typing effect).
5. Everything is automatically saved to the `conversations/` folder on disk.

Special Features

- **@-Mentions**: You can explicitly choose which agents should reply by tagging them with the `@` symbol (e.g., `@Alice what do you think?`). The system bypasses normal probability rules and guarantees only the tagged agents respond.
- **Image Generation**: The simulation includes a dedicated image-generation agent named "Picasso" (role `image_generator`). The orchestrator never selects it by itself, so you can only generate images by mentioning it, e.g. `"@Picasso generate an image of..."`. The image is rendered directly in the chat window via the configured image generation API (`image_api`); click a generated image to save it as your final poster.

Project Structure

File	Description
`main.py`	Launches the Gradio web interface and contains only the mechanics: login, screen navigation, answer checking, tutorial logic, condition assignment, live streaming, pause/resume, poster saving and event logging. It contains no wording; every text comes from `config.yaml`
`config.yaml`	**The main file for editing the app** -- all screen text (`screens`), UI settings (`ui`), agents and their prompts (`agents`), orchestrator settings and prompt (`orchestrator`), shared agent prompt (`scenario`), API, logging and Google Sheets settings
`agents.py`	Agent data types, conversation state, and LLM response generation, including the image-generating agent (stable v0.5 logic)
`orchestrator.py`	Decides which agents respond and when, using an AI model and `@mentions` (stable v0.5 logic)
`simulation.py`	Core loop connecting agents, orchestrator, user input, and logging

File	Description
`config_loader.py`	Reads and validates `config.yaml` into typed Python objects and gives `main.py` access to the `screens` texts
`conversation_logger.py`	Saves conversation transcripts and study-flow events to `.txt` (and optionally `.json`) files
`initial_prompt.txt`	A sample opening message you can copy-paste to start a conversation
`multi_agent_chat_prototype_v1.html`	Static design prototype of the study screens (reference only; not used by the app)
`run_chatbot.sh`, `test_gsheets.py`	Helper scripts for weekly restarts and for testing the Google Sheets connection

---

Prerequisites

- **Python 3.10+**
- An **OpenAI-compatible API endpoint** (OpenAI, vLLM, Ollama, TGI, etc.)
- A valid API key for that endpoint

Quick Start

```
pip install -r requirements.txt
python main.py
```

Open the URL printed in the terminal (e.g. `http://127.0.0.1:7860`) in your browser.

To use a different configuration file:

```
python main.py --config path/to/custom_config.yaml
```

Configuration Guide

Everything is controlled by `config.yaml`. Below is a plain-language tour of every section, what each setting does, and what happens when you change it.

API

```
api:
  base_url: "https://api.openai.com/v1"
  api_key: "sk-..."
  organization: null
  timeout: 60
  agent_model: "gpt-4o"
  orchestrator_model: "gpt-4o-mini"
```

Setting	What it does
`base_url`	The address of the AI model server. Use `https://api.openai.com/v1` for OpenAI, or the address of your local model (e.g. Ollama).
`api_key`	Your secret key for that API. Keep this private.
`organization`	Optional organization ID (OpenAI-specific; ignored by most other APIs). Use `null` if not needed.
`timeout`	How many seconds to wait before giving up on a slow model response. Increase this (e.g. to `120`) if you see timeout errors.

Setting	What it does
`agent_model`	The AI model used by every agent to generate their chat replies. A smarter model (e.g. `gpt-4o`) produces more natural, nuanced responses but costs more and is slightly slower.
`orchestrator_model`	The AI model used to decide *who* speaks next. This task is simpler, so a smaller/faster model (e.g. `gpt-4o-mini`) works well and is cheaper.

Image API

```
image_api:
 base_url: "https://api.openai.com/v1"
 api_key: "sk-..."
 model: "gpt-image-1"
 dummy_image_url: "https://..."
```

Setting	What it does
`base_url` / `api_key`	Endpoint and key of the image-generation API used by the agent with the `image_generator` role.
`model`	The image model used to generate images.
`dummy_image_url`	Image shown instead of a generated one when the image API is not reachable (useful for testing without an image API key).

Scenario

```
ui:
 task_description: |
   In this task, you will collaborate with four AI agents to design a ...
scenario:
 agent_extra_system_prompt: >-
   The user's name is {user_name}. ...
```

Setting	What it does
`ui.task_description`	The task that all agents read as background context (it is appended to every agent's prompt and added to the conversation as the first system message). It is also shown on the Briefing screen (one paragraph per line), in the Task panel of the live task and, if `ui.show_task_description` is `true`, on the login screen. Change this to completely change the simulation setting (e.g. a hospital team meeting, a university seminar, a startup pitch).
`scenario.agent_extra_system_prompt`	Extra instructions appended to *every* agent's personality. For example, it currently tells agents to keep language accessible and not answer questions directed at you. Edit this to globally tweak agent behaviour without touching each agent individually.

Example -- changing the scenario:

Replace the `ui.task_description` with:

```
task_description: |
 You are a medical student presenting a clinical case to a group of supervising
 doctors. The team will question your diagnosis and treatment plan. Show your
 reasoning clearly.
```

The agents will start working on this new task. Also adapt the matching texts in the `screens` section (for example the check questions and the Task "Goal" text) and the agents' own prompts.

Agents

```
agents:
 - name: "Alice"
   role: "supporter"
   title: "senior engineer"
   description: "Specializes in advertising strategy and audience insights..."
   card_description: "Provides a strategic framework for the campaign."
   chat_color: "#d4f5d4"
   talkativeness: 0.75
   temperature: 0.8
   max_tokens: 256
   system_prompt: >
     You are Alice, a supportive senior engineer...
```

Each agent entry controls one character. You can add as many agents as you like, or remove any of the defaults.

Setting	What it does
`name`	The agent's display name in the chat. Must be unique. If it matches your login name, the app renames it automatically to avoid confusion.
`role`	Guides the orchestrator on when to pick this agent. See the role table below.
`title`	Short job title shown after the name in parentheses in the chat (e.g. "Alice (senior engineer)"), and as the subtitle of the agent cards. Purely cosmetic.
`description`	Longer description of what the agent does. Shown on the agent cards (Meet team screen and the Agents panel of the live task) unless a `card_description` is given.
`card_description`	Optional shorter description used on the agent cards, so that all cards have a similar length (the Check 2 questions state that the descriptions are of approximately equal length). Remove it to show the full `description`.
`chat_color`	Background colour of this agent's message bubble. Any CSS colour works (e.g. `"#d4f5d4"`, `"lightblue"`).
`talkativeness`	A number from `0.0` to `1.0`. Higher = more likely to jump into the conversation unprompted. An agent that is directly addressed always responds regardless of this value.
`temperature`	How "creative" or varied the replies are. `0.0` = very consistent and focused. `1.0`+ = more spontaneous and varied. Values around `0.7`-`0.9` feel natural.
`max_tokens`	The maximum length of a single reply, measured in tokens (roughly ¾ of a word each). `128` keeps replies short. `512` allows longer, more detailed responses.
`system_prompt`	The agent's personality description -- who they are, how they speak, and any hard rules. This is the most powerful thing to customise.

Example -- making an agent quieter:

```
- name: "Bob"
  talkativeness: 0.2   # Was 0.6 -- Bob will now rarely speak unless addressed
  temperature: 0.5     # Was 0.9 -- replies will be more predictable
  max_tokens: 128      # Was 256 -- replies will be shorter
```

Example -- adding a new agent:

```
- name: "Priya"
  role: "mediator"
  title: "project manager"
  chat_color: "#fde8f5"
  talkativeness: 0.6
  temperature: 0.75
  max_tokens: 256
  system_prompt: >
```

```
  You are Priya, a calm and experienced project manager. You keep discussions
  on track, summarise points of agreement, and gently redirect when the
  conversation gets stuck. You speak in short, clear sentences.
```

Agent Roles

The `role` field tells the orchestrator *when* to prioritise an agent:

Role	When this agent is more likely to speak
`challenger`	When there are unexplored alternatives or a decision seems rushed
`comedian`	When the mood needs lightening with humor
`creative`	When generating novel ideas and brainstorming
`devil_advocate`	When specifically challenging rapid consensus
`expert`	When deep domain knowledge and facts are needed
`image_generator`	Reserved for generating images (e.g., Picasso)
`manager`	When driving next steps and action items
`mediator`	When participants disagree or the conversation gets tense
`quiet_observer`	Almost never -- only when directly asked a question
`skeptic`	When someone makes a bold claim or proposal that needs scrutiny
`summarizer`	When concluding long debates and aligning the team
`supporter`	When someone shares a good idea or needs encouragement
`user_proxy`	When evaluating ideas from the end-user's point of view

Orchestrator

The orchestrator is the invisible "director" that reads the conversation after each message and decides which agents speak next. You rarely need to change these settings, but here is what they do:

```
orchestrator:
 max_agent_turns: 4
 max_orchestrator_rounds: 1
 min_responders: 1
 response_threshold: 0.4
 high_relevance_threshold: 0.75
 base_vocal_factor: 0.7
 temperature: 0.3
 max_tokens: 200
 context_window: 15
 agent_context_window: 40
 system_prompt: |
   You are a conversation orchestrator ...
```

Setting	What it does	Effect of increasing
`max_agent_turns`	Max number of agents that can reply per orchestrator round. Together with `max_orchestrator_rounds` it sets the maximum number of agent replies per user message (`max_agent_turns x max_orchestrator_rounds`)	More agents reply per message (chattier)
`max_orchestrator_rounds`	How many times the orchestrator can run per user message (agents can respond to each other)	More back-and-forth among agents before the conversation returns to you
`min_responders`	Minimum number of agents guaranteed to reply to each message	Prevents awkward silences; raise to 2 for a livelier discussion

Setting	What it does	Effect of increasing
`response_threshold`	An agent is selected if its relevance score is at or above this value (0-1).	Lower = more agents reply to everything; higher = only the most relevant agents speak
`high_relevance_threshold`	If an agent's relevance score meets this, they always reply regardless of talkativeness	Lower = more agents are considered "important enough" to override talkativeness
`base_vocal_factor`	A global multiplier that dampens how often agents volunteer to speak	Lower = quieter simulation overall
`temperature`	How varied the orchestrator's decisions are	Higher = less predictable turn-taking
`max_tokens`	Maximum length of the orchestrator's answer (its JSON decision)	Rarely needs changing
`context_window`	How many recent messages the orchestrator reads when deciding	Higher = longer memory for context, slower decision
`agent_context_window`	How many recent messages each agent reads when writing a reply	Higher = agents remember more of the conversation, slower and costlier replies
`system_prompt`	The instruction text sent to the orchestrator model. Placeholders: `{user_name}`, `{last_speaker}`, `{response_threshold}`, `{high_relevance_threshold}`, `{max_agent_turns}`, `{min_responders}`; use `{{` and `}}` for literal braces	Edit to change how the orchestrator weighs roles and patience rules

Example -- creating a very focused, one-at-a-time conversation:

```
orchestrator:
 max_agent_turns: 1
 max_orchestrator_rounds: 1
 min_responders: 1
 response_threshold: 0.6
 base_vocal_factor: 0.5
```

Only the single most relevant agent speaks per message, and agents are generally less eager to jump in.

Example -- creating a lively, multi-voice discussion:

```
orchestrator:
 max_agent_turns: 3
 max_orchestrator_rounds: 2
 min_responders: 2
 response_threshold: 0.3
 base_vocal_factor: 0.9
```

Up to three agents reply per round, the orchestrator runs twice so agents can respond to each other, and at least two agents are guaranteed to reply to every message.

Screens (all study-screen text)

Everything the participant reads in the study flow is defined in the `screens` section of `config.yaml`. To change the wording of a screen, edit it there; no change in `main.py` is needed.

```
screens:
 common:        # "Previous" and "Continue" captions
 topbar:        # progress-bar labels (Consent, Briefing, Check 1, ...)
 event_log:     # caption of the event-log drawer
 login:         # field labels, placeholders, button, error messages
 consent:       # title, paragraphs, checkbox label, warning
 briefing:      # title and body (uses {task_description})
 check1:        # title, intro, buttons, result messages, questions + answers
 team:          # title, intro, note, shuffle button, coordinator agent name
 check2:        # same structure as check1
 tutorial:      # title, send button, and the texts of the 4 practice steps
```

```
manipulation:  # title, reflection texts, conditions 1-4
check3:        # title, intro, scale labels, messages, Likert items
live:          # welcome text, panel titles, tips, buttons, poster messages
debrief:       # title, paragraphs, link text, prototype note
```

What you want to change	Where
Button captions ("Previous", "Continue", "Check answers", "Send practice message", "Pause", "Resume", "Send", "Finish task", ...)	`screens.common`, and the `*_button` keys of each screen
A check question or its answers	`screens.check1.questions` / `screens.check2.questions` (each question has an `id`, a `title`, `options` with an `id` and `text`, the `correct` option id, an optional `shuffle` (true/false, default true) for shuffling that question's answers, and an optional `layout` (`vertical` = top to bottom, `horizontal` = left to right; default `vertical`); the number of questions can be changed)
The Likert items or the scale labels	`screens.check3.items` (each with an `id`, `text` and `layout`, default `horizontal`) / `screens.check3.scale`
The leadership-role conditions	`screens.manipulation.conditions` (`reflection: true` requires a written reflection)
The tutorial instructions and the agent to @mention	`screens.tutorial.step1` ... `step4`
Which agent is shown on top of the team page	`screens.team.coordinator_agent`

Notes: - Keys that end in `_html`, and the `paragraphs` and `body_html` entries, may contain simple inline HTML such as `<strong>...</strong>`. All other text is shown as plain text. - Some texts contain `{placeholders}` (for example `{user_name}`, `{max_length}`, `{min_length}`, `{count}`); the available ones are listed in the comments next to the entry in `config.yaml`. - Agent names, titles and descriptions on the team page and in the live task come from the `agents` section, not from `screens`.

Logging

```
logging:
  output_dir: "conversations"
  save_json: true
```

Setting	What it does
`output_dir`	Folder where transcripts are saved. Each session creates a new file named with the (UTC) date and time (e.g. `2025-06-01_14-30-00.txt`). The transcript also contains the study-flow events.
`save_json`	If `true`, a machine-readable `.json` file is saved alongside the text file. Useful if you want to analyse conversations programmatically.

UI

```
ui:
  title: "Multi-Agent Leadership Simulation"
  chatbot_height: 500
  task_panel_height: 560
  task_panel_width: 18
  agents_panel_height: 340
  agents_panel_width: 18
  tips_panel_height: 200
  tips_panel_width: 18
  server_port: 7860
  share: false
  password: "admin"
  live_typing_delay: 0.12
  true_delay_min: 0.0
  true_delay_max: 0.0
  user_typing_pause_seconds: 0.0
  input_placeholder: "Type your message..."
```

```
max_username_length: 16
user_chat_color: "#e9eefc"
show_task_description: false
task_description_title: "■ Task Description"
task_description: |
  In this task, you will collaborate with four AI agents ...
greetings: ["Hello", "Hi", "Hola", ...]
pause_border_color: "rgba(255, 165, 0, 1.0)"
pause_animation_duration_seconds: 1.5
backup_names: ["Eve", "Frank", ...]
show_prototype_note: true
min_reflection_length: 20
exit_survey_url: "https://example.qualtrics.com/jfe/form/SV_demo?participant_id={session_id}"
```

Setting	What it does
`title`	The heading shown at the top of the page.
`chatbot_height`	Height of the chat window in pixels. Increase for a taller window.
`task_panel_height` / `task_panel_width`	Height (in pixels, or a CSS string like `"560px"`) and width (as a percentage of the row, or a CSS string like `"18%"` / `"260px"`) of the "Task" panel on the Live task screen.
`agents_panel_height` / `agents_panel_width`	Height (in pixels or CSS string) and width (percentage or CSS string) of the "Agents" panel on the Live task screen.
`tips_panel_height` / `tips_panel_width`	Height (in pixels or CSS string) and width (percentage or CSS string) of the "Tips" panel on the Live task screen. The middle chat column automatically fills the remaining width (`100% - task_panel_width - max(agents_panel_width, tips_panel_width)`).
`user_chat_color`	Background colour of the participant's own messages in the live chat (agents use their own `chat_color`).
`server_port`	The local port the web server runs on. Change if port 7860 is already in use.
`share`	If `true`, Gradio creates a temporary public link so others can access your simulation from anywhere. Useful for remote studies.
`password`	A password required at login. Leave blank (`""`) to disable.
`live_typing_delay`	Seconds between each word chunk appearing as the agent "types". `0.03` = very fast. `0.15` = slow, human-like. Default `0.12` feels natural.
`true_delay_min` / `true_delay_max`	Random wait (in seconds) before an agent starts typing. Simulates a realistic "thinking" pause. Both `0.0` means agents start instantly.
`user_typing_pause_seconds`	How long agents wait after you clear the text box (without sending) before they continue speaking.
`max_username_length`	Maximum number of characters allowed in the login name.
`show_task_description`	Whether the task description box is shown on the login screen (`true`/`false`). It does not affect the Briefing screen or the live task. Currently `false`.
`task_description_title`	Heading of the task description box on the login screen (only used if `show_task_description` is `true`).
`task_description`	The task text; see Scenario.
`greetings`	List of random welcome phrases shown after you log in.
`pause_border_color`	Colour of the glowing border that appears when you click Pause. Any CSS colour works.
`pause_animation_duration_seconds`	How fast the pause glow pulses (in seconds per cycle).
`input_placeholder`	Placeholder text of the message box in the live chat.

Setting	What it does
`backup_names`	Names used to rename an agent whose name equals the participant's name.
`show_prototype_note`	Whether the "Prototype note" boxes are shown on the Role and Debrief screens.
`min_reflection_length`	Minimum number of characters of the written reflection (for the conditions that require one).
`exit_survey_url`	Link on the Debrief screen. `{session_id}` is replaced by the session ID of the participant.

Example -- making the typing feel more human:

```
live_typing_delay: 0.08        # Slightly faster typing
true_delay_min: 0.5            # Wait at least 0.5s before starting to type
true_delay_max: 2.0            # Wait up to 2s (random per agent)
```

This makes agents pause before responding and then type at a moderate speed, which feels much more like a real conversation.

Google Sheets Integration (optional)

```
google_sheets:
 spreadsheet_id: "..."
 sheet_name: "Sheet1"
 cell: "A1"
 credentials_file: "service_account.json"
```

When `share: true`, the public Gradio URL is automatically written to the specified cell in your Google Sheet. This is useful for research studies where you need to share the current link with participants. Gradio share links last for up to 7 days, so for long-running deployments, you must schedule a restart every week.

Setup: 1. `pip install gspread` 2. Create a Google Cloud service account and download its JSON key. 3. Share your spreadsheet with the service account's email address `client_email` (give it "Editor" permissions). If you do not do this, you will get a `403 The caller does not have permission` error. 4. Fill in the fields above.

Weekly Automatic Restart (Linux/Raspberry Pi): Because Gradio share links expire after 7 days, you can automate a weekly restart using Cron so your Google Sheet always has a fresh, working link.

1. Ensure the `run_chatbot.sh` script in this repository has the correct paths for your conda environment and project directory.
2. Make the script executable: `bash chmod +x run_chatbot.sh`
3. Open your crontab editor: `bash crontab -e`
4. Add the following line to the bottom to restart the server every Sunday at midnight (adjust paths as needed): `bash 0 0 * * 0 /path/to/multi_agent_chat/run_chatbot.sh`

Troubleshooting

Problem	Solution
`FileNotFoundError: config.yaml`	Run `python main.py` from the project's root folder
API timeout errors	Increase `api.timeout` (e.g. to `120`) or check your API endpoint is reachable
Agents repeat each other	Lower each agent's `temperature` or reduce `orchestrator.max_agent_turns`
Only one agent ever responds	Increase `orchestrator.min_responders` to `2`, or raise agents' `talkativeness` values
Agents respond too slowly	Set `true_delay_min` and `true_delay_max` to `0.0`, or use a faster model

Problem	Solution
Agents type too fast / too slow	Adjust `ui.live_typing_delay` -- try `0.05` for fast or `0.15` for slow
`KeyError: Missing entry 'screens....'` at start-up	A text entry was deleted or misspelled in the `screens` section of `config.yaml`; restore the entry named in the message
An agent keeps answering questions directed at you	Strengthen the `agent_extra_system_prompt` (add explicit instructions to stay silent when you are addressed), or lower that agent's `talkativeness`
Port 7860 is already in use	Change `ui.server_port` to any free port, e.g. `7861`

Practical Guide for Study Hosts

This section explains common, everyday situations for researchers and study coordinators who host the **Interaction Portal** (inside the `interaction_portal` folder) without needing a programming background.

---

1. Automatic Startup Self-Check in the Console

Whenever you start the portal from the `interaction_portal` folder with `python -m app.main`, the application automatically runs a fast, 2-second self-check of all 14 internal test suites before opening the website:

- **Green Checkmark (`✓ Startup self-check: 14/14 test suites passed`)**: Everything is configured properly and ready for participants.
- **Orange Warning (`■ Startup warning: ...`)**: If a configuration file or folder has an issue, the console prints a calm orange warning showing which check had an issue, one sentence explaining why, and one sentence explaining how to fix it. **The portal will still start** so your study is not interrupted.
- **Optional Skip**: If you ever want to skip this 2-second check on startup, run `PORTAL_SKIP_STARTUP_TESTS=1 python -m app.main`.

---

2. Setting the Maximum Number of Concurrent Participants (`max_concurrent_participants`)

What this setting does:

In `interaction_portal/config/config.yaml` (under the `server:` section), the setting `max_concurrent_participants` controls how many people are allowed to work inside the portal at the exact same time. You can set this number to `30`, `100`, `300`, or any number that fits your study wave. You can also override it when starting the server by setting `PORTAL_MAX_PARTICIPANTS=300`.

What happens when the portal reaches full capacity:

Suppose you set `max_concurrent_participants: 100`, and 100 people are currently inside the portal working on their tasks. If a 101st person clicks the entry link from Qualtrics or Prolific:

- The portal will **not** crash or invalidate their link.
- Instead, the 101st person will see a polite waiting screen explaining that the study portal is currently at full capacity (100 active participants) and asking them to wait a moment and refresh the page.
- As soon as one of the active participants finishes or leaves, the 101st person can refresh and start their session normally.

Advantages (Pros) of setting a higher limit (such as 100 or 300):

- **Faster data collection**: You can open a large wave of slots on Prolific or Qualtrics at the same time and finish collecting data in a few hours.
- **Automatic internal scaling**: When you increase `max_concurrent_participants`, the portal automatically allocates more background worker threads and network connections so participants do not wait in line behind one another on your server.

Disadvantages and Warnings (Cons) of setting a higher limit (such as 100 or 300):

- **External Artificial Intelligence (AI) Provider Rate Limits**: In the AI conditions, every time one participant sends a message to the team, up to 5 AI agents process that message. For example, if 300 participants are active at the same time and each person sends 1 message per minute, your server will make up to **1,500 requests per minute** to your **Large Language Model (LLM)** (the AI text generator, such as OpenAI ChatGPT or Anthropic Claude). If your AI provider account is on a basic starter tier, the

AI company will temporarily block requests when too many arrive in the same minute.

- **Practical rule for AI accounts**: Before running 100 to 300 concurrent participants with a live AI model, check your usage tier in your OpenAI, Anthropic, DeepSeek, GLM, or Qwen account dashboard to make sure your account allows at least 1,500 to 3,000 requests per minute.
- **Server Memory and Processor Capacity**: Hosting 300 simultaneous users requires more computer working memory, known as **Random-Access Memory (RAM)**, than hosting 30 users. Always run the built-in simulation stress test (described below) on your hosting computer before launching a large wave.

---

3. Running a Safe Simulation Stress Test (`stress_test.py`)

What it is:

Before inviting real participants (and before spending money on participant payments or AI usage), you can run a simulated stress test. The computer creates 30, 100, or 300 virtual participants that enter the portal at the exact same time, complete the practice steps, chat with the AI team, edit their campaign cards, and submit their check-in answers. It simulates realistic AI waiting times (for example, 500 to 1,000 milliseconds per call) using the offline mock model so it costs **zero money**.

How to run it from your terminal (inside the `interaction_portal` folder):

```
# Test 30 concurrent participants
python -m app.tools.stress_test --participants 30 --llm-delay-ms 500

# Test 100 concurrent participants
python -m app.tools.stress_test --participants 100 --llm-delay-ms 500

# Test 300 concurrent participants
python -m app.tools.stress_test --participants 300 --llm-delay-ms 1000
```

How to read the result:

At the end of the test, look for `Participants Completed: 100.0%` and `Stress Test Verdict: PASSED`. If it passes, your computer and database are ready for that number of concurrent participants.

---

4. Helping a Participant Who Closed Their Browser or Lost Internet

- **Same browser within 15 minutes (Automatic Recovery)**:

If a participant accidentally closes their browser tab, refreshes the page, or loses internet briefly, they can simply click their original entry link again in the **same web browser** within 15 minutes (`resume_gap_seconds: 900` in `interaction_portal/config/config.yaml`). Because their browser holds a secure session cookie and all countdown timers are saved on the server, they will automatically return to the exact screen and remaining time where they left off.

- **Different browser or cleared cookies (Manual Host Recovery via Resume URL)**:

To stop two different people from sharing the same **Participant Identifier (PID)** (the unique participant code in the link, such as `pid=P101`), the portal blocks anyone who opens an already-started PID from a different browser without the session cookie. If a real participant emails you because their browser crashed and cleared their cookies, you can give them their private recovery link:

1. Open the administrator dashboard in your browser at `http://127.0.0.1:8000/admin` (or `https://your-domain.com/admin` if hosted online) and sign in with your `admin_token` (see Section 6 below).
2. Look at the **Recent Sessions** table at the bottom of the page and find the participant's PID.
3. Right-click and copy the **Resume URL** (`http://127.0.0.1:8000/?resume=...`) shown in the **Recovery** column and send that link to the participant. Clicking that link restores their session cookie and lets them continue from where they stopped.

---

5. Running the Pre-Flight Readiness Check (`preflight.py`)

What it is:

Before you publish your study link to real participants, run the pre-flight check from inside the `interaction_portal` folder. In less than 2 seconds, it verifies that your configuration files, all 10 study conditions, all 7 client briefs, all specialist private files, the database, and your AI provider key are properly set up.

```
python -m app.tools.preflight
```

Look for `Verdict: READY` at the bottom of the output.

6. Opening the Admin Dashboard (`http://127.0.0.1:8000/admin`) and Downloading Spreadsheets

Where to find the Admin Token:

The administrator dashboard is protected by a password called `admin_token` so that participants cannot see study statistics or download data.

- **Default token for local testing**: In `interaction_portal/config/config.yaml` (under the `testing:` section), the default token is: `admin-test-token`
- **Custom token for live studies**: Before putting the server online for real participants, change `admin_token` in `interaction_portal/config/config.yaml` (or set the environment variable `PORTAL_ADMIN_TOKEN`) to a long secret password of at least 24 characters.

How to open the Admin Dashboard in your web browser:

1. Make sure the portal is running (`python -m app.main`).
2. Open your web browser and go to: `http://127.0.0.1:8000/admin` *(Or go directly to `http://127.0.0.1:8000/admin?token=admin-test-token`, replacing `admin-test-token` with your own token if you changed it.)*
3. Type your `admin_token` into the **Portal Admin Login** box and click **Sign In**. Your browser will save a login cookie for 1 hour so you can click any export button on the page.

What you can do inside the Admin Dashboard:

- **Study Overview**: Shows how many sessions are Total, Completed, Active right now, or Dropped/Timed out, alongside your configured `max_concurrent_participants` capacity.
- **Condition Balance Table**: Shows how many participants have been assigned to each of the 10 experimental conditions (`PAS-COORD` through `NOAI`) and how many have finished.
- **Flat Data Exports**: Click any blue button on the dashboard to download **Comma-Separated Values (CSV)** spreadsheet files that open directly in Microsoft Excel, R, Stata, SPSS, or Python:
- **Task Summary (Wide CSV)** (`http://127.0.0.1:8000/admin/export/task_summary.csv`): The easiest file for statistical analysis. Each row represents one participant's work on one client task, with pre-calculated totals for messages sent, `@mention` messages, broadcast messages, hidden items uncovered, panel views, card completion, raw and strict typed share, and post-task check-in answers (`ci1` to `ci5`).
- **Events CSV, Panels CSV, Transcripts CSV, Card Spans CSV, Check-ins CSV, and Sessions CSV**: Detailed action-by-action logs for fine-grained process analysis.

7. Backing Up Data and Removing Personal Information (`backup.py` and `purge_pii.py`)

- **Creating a safe backup while the server is running**:

Run the following command in your terminal from inside the `interaction_portal` folder:

```
python -m app.tools.backup --dest backups/ --keep 14
```

This creates a verified, compressed copy of the database in the `backups/` folder without interrupting participants who are currently taking the study.

- **Scanning for and removing Personally Identifiable Information (PII)**:

Although the orientation page instructs participants not to type personal details, a participant might still type their email address, phone number, or name into the chat or onto a card. Run the following commands from inside the `interaction_portal` folder to scan the database and replace any detected personal details with `[REDACTED_...]` tags:

```
# Preview any detected personal information without changing the database
python -m app.tools.purge_pii --dry-run

# Redact detected personal information across transcripts, events, and cards
python -m app.tools.purge_pii
```